Compiler Design Laboratory Report

Lexical, Syntax, and Semantic Analysis

Student Name
Roll Number
Department of Computer Science

December 12, 2025

Contents

1 Introduction

Compiler design is a multi-phase process that transforms high-level source code into executable machine code. The front-end of a compiler consists of three critical phases: lexical analysis, syntax analysis, and semantic analysis. Each phase plays a distinct role in understanding and validating the structure and meaning of the source code.

1.1 The Three Phases of Compiler Front-End

Lexical Analysis (scanning) is the first phase where the source code is read as a stream of characters and converted into a sequence of tokens. Tokens are the smallest meaningful units such as keywords, identifiers, operators, and literals. This phase removes comments and whitespace, simplifying the input for subsequent phases.

Syntax Analysis (parsing) verifies that the token sequence conforms to the grammatical rules of the programming language. It constructs a parse tree or abstract syntax tree (AST) that represents the hierarchical structure of the program. This phase detects syntactic errors such as missing semicolons, unmatched parentheses, or incorrect statement ordering.

Semantic Analysis examines the parse tree to ensure that the program makes logical sense. It performs type checking, verifies variable declarations, checks scope rules, and ensures that operations are performed on compatible data types. This phase catches errors that are syntactically correct but semantically meaningless, such as adding a string to an integer.

1.2 Importance of Multi-Phase Analysis

The separation of compilation into distinct phases provides several advantages:

- **Modularity:** Each phase has a well-defined responsibility
- **Error Detection:** Different types of errors are caught at appropriate stages
- **Maintainability:** Changes to one phase don't affect others
- **Reusability:** Components can be reused across different compilers
- **Optimization:** Each phase can be independently optimized

1.3 Experiment Context

This laboratory exercise demonstrates all three phases of compiler front-end analysis on a simple code snippet featuring variable declaration, type specification, conditional statements, and control flow. The code uses a custom syntax that combines elements from various programming languages:

```
let i as DOUBLE;
if (i == 3) begin stop;
end
```

This experiment will showcase how a compiler processes this code through lexical tokenization, syntactic validation, and semantic type checking, ultimately detecting a type mismatch error between a DOUBLE variable and an integer constant.

2 Objective

The objectives of this laboratory exercise are to:

1. Perform **lexical analysis** to tokenize the given code snippet
2. Perform **syntax analysis** to validate grammatical structure
3. Perform **semantic analysis** to check type compatibility
4. Understand the integration of all three compiler phases
5. Implement symbol table management for tracking variable declarations
6. Demonstrate error detection and reporting mechanisms

3 Theory

3.1 Lexical Analysis

Lexical analysis transforms a character stream into a token stream. Each token has:

- **Token Type:** Category (keyword, identifier, operator, etc.)
- **Lexeme:** Actual text matched
- **Attributes:** Additional information (value, type, etc.)

Regular Expressions are used to specify token patterns. For example:

- Identifier: `[a-zA-Z][a-zA-Z0-9]*`
- Integer: `[0-9]+`
- Keyword: Exact string matches like "if", "while", "let"

3.2 Syntax Analysis

Syntax analysis uses **context-free grammars** (CFG) to define language structure. A CFG consists of:

- **Terminals:** Tokens from lexical analysis
- **Non-terminals:** Abstract syntactic categories
- **Production Rules:** Define how non-terminals expand
- **Start Symbol:** Root of the parse tree

Bison generates **LALR(1) parsers** that use a bottom-up parsing strategy, reducing input tokens according to grammar rules until reaching the start symbol.

3.3 Semantic Analysis

Semantic analysis performs several key tasks:

- **Symbol Table Management:** Tracking declared variables and their types
- **Type Checking:** Ensuring operations are performed on compatible types
- **Scope Resolution:** Verifying variable accessibility
- **Declaration Checking:** Ensuring variables are declared before use

Type checking rules might include:

- Arithmetic operations require numeric operands
- Comparison operators require compatible types
- Assignment requires type compatibility between LHS and RHS

4 Problem Statement

Given the following code snippet:

```
let i as DOUBLE;
if (i == 3) begin stop;
end
```

Listing 1: Input code for analysis

Tasks:

1. Perform lexical analysis to identify all tokens
2. Perform syntax analysis to validate the structure
3. Perform semantic analysis to check type compatibility
4. Detect the type mismatch between DOUBLE variable and integer constant

5 Implementation

5.1 Input File (input.txt)

```
let i as DOUBLE;
if (i == 3) begin stop;
end
```

Listing 2: Complete input file

5.2 Lexical Analyzer (lexer.l)

```
%option noyywrap
%{
    #include <stdio.h>
    #include <stdlib.h>
    #include <string.h>
    #include "parser.tab.h"

    int lineno = 1; // initialize to 1
    void yyerror(char *s);
%}
alpha      [a-zA-Z]
digit      [0-9]
alnum      {alpha}|{digit}
print      [ -~]
ID         {alpha}{alnum}*
ICONST     [0-9]{digit}*
FCONST     {digit}*"."{digit}+
CCONST     (\'{print}\')
%%
"//".* { }
"int"        { return INT; }
"DOUBLE"     { return DOUBLE; }
"float"      { return DOUBLE; }
"char"       { return CHAR; }
"let"        { return LET; }
"FOR"        { return FOR; }
"TO"         { return TO; }
"DO"         { return DO; }
"as"         { return AS; }
"begin"      { return BEGINN; }
"end"        { return ENDD; }
"stop"       { return STOP; }
"if"         { return IF; }
"else"       { return ELSE; }
"+"        { return ADDOP; }
"-"        { return SUBOP; }
"*"        { return MULOP; }
"/"        { return DIVOP; }
"=="       { return EQUOP; }
">"        { return GT; }
"<"        { return LT; }
"("        { return LPAREN; }
")"        { return RPAREN; }
"{"        { return LBRACE; }
"}"        { return RBRACE; }
";"        { return SEMI; }
"="        { return ASSIGN; }
{ID}         {strcpy(yylval.str_val, yytext); return ID;}
{ICONST}     {return ICONST;}
```

```
{FCONST}    {return FCONST;}
{CCONST}    {return CCONST;}
"\n"        { lineno += 1; }
[ \t\r\f]+
.       { yyerror("Unrecognized character"); }
%%
```

Listing 3: Flex specification for lexical analysis

5.3 Symbol Table Implementation (symtab.c)

This file is included by the parser to handle variable tracking and type checking.

```
#include <stdio.h>
#include <string.h>

#define INT_TYPE 1
#define DOUBLE_TYPE 2
#define CHAR_TYPE 3
#define UNDEF_TYPE -1

struct Symbol {
    char name[100];
    int type;
} symtab[100];

int sym_count = 0;

void insert(char* name, int type) {
    // Check if already exists
    for(int i=0; i<sym_count; i++) {
        if(strcmp(symtab[i].name, name) == 0) {
            printf("Error: Variable %s already declared\n", name);
            return;
        }
    }
    strcpy(symtab[sym_count].name, name);
    symtab[sym_count].type = type;
    sym_count++;

    char* type_name = (type == DOUBLE_TYPE) ? "DOUBLE_TYPE" : "INT_TYPE";
    printf("In line no %d, Inserting %s with type %s in symbol table.\n", lineno, name, type_name);
}

int idcheck(char* name) {
    for(int i=0; i<sym_count; i++) {
        if(strcmp(symtab[i].name, name) == 0) {
            return 1;
        }
```

```
    }
    printf("Error: Variable %s not declared at line %d\n", name,
        lineno);
    return 0;
}

int gettype(char* name) {
    for(int i=0; i<sym_count; i++) {
        if(strcmp(symtab[i].name, name) == 0) {
            return symtab[i].type;
        }
    }
    return UNDEF_TYPE;
}

int typecheck(int type1, int type2) {
    if(type1 == type2) return 1;

    char* t1 = (type1 == DOUBLE_TYPE) ? "DOUBLE_TYPE" : "INT_TYPE
        ";
    char* t2 = (type2 == DOUBLE_TYPE) ? "DOUBLE_TYPE" : "INT_TYPE
        ";

    printf("In line no %d, Data type %s is not matched with Data
        type %s.\n", lineno, t1, t2);
    return 0;
}
```

Listing 4: Symbol Table Logic

5.4 Parser and Semantic Analyzer (parser.y)

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Initialize lineno
extern int lineno;
extern int yylex();
void yyerror(char *s);

// Include the symbol table logic defined above
#include "symtab.c"

int error_count = 0;
%}

%union{
    char str_val[100];
    int int_val;
```

```
}

%token INT IF ELSE WHILE CONTINUE BREAK PRINT DOUBLE CHAR
%token ADDOP SUBOP MULOP DIVOP EQUOP LT GT
%token LPAREN RPAREN LBRACE RBRACE SEMI ASSIGN
%token LET AS BEGINN ENDD STOP
%token<str_val> ID
%token ICONST FCONST CCONST

%type<int_val> type constant exp

%left EQUOP
%left LT GT
%left ADDOP SUBOP
%left MULOP DIVOP

%start code

%%
code: statements
    ;
statements: statements statement
    |
    ;
statement: let_declaration
    | if_statement
    | begin_block
    | STOP SEMI
    ;
let_declaration: LET ID AS type SEMI
    {
        insert($2, $4);
    }
    ;
begin_block: BEGINN statements ENDD
    ;
type: DOUBLE
    {
        $$ = DOUBLE_TYPE;
    }
    | INT
    {
        $$ = INT_TYPE;
    }
    ;
exp: constant
    {
        $$ = $1;
    }
    | ID
    {
```

```
        if(idcheck($1)==0)
        {
            error_count++;
            $$ = UNDEF_TYPE;
        }
        else
        {
            $$ = gettype($1);
        }
    }
    | exp EQUOP exp
    {
        if(typecheck($1, $3)==0)
        {
            error_count++;
        }
        $$ = INT_TYPE; // Boolean result is effectively int
    }
    | LPAREN exp RPAREN
    {
        $$ = $2;
    }
    ;
constant: ICONST
    {
        $$ = INT_TYPE;
    }
    | FCONST
    {
        $$ = DOUBLE_TYPE;
    }
    ;
if_statement: IF LPAREN exp RPAREN statement
    ;
%%

void yyerror(char *s)
{
    printf("Syntax error at line %d: %s\n", lineno, s);
    error_count++;
}

int main(int argc, char *argv[])
{
    // If you want to read from file directly via C code:
    // freopen("input.txt", "r", stdin);
    // freopen("output.txt", "w", stdout);

    yyparse();

    if(error_count == 0)
```

```
    {
        printf("\nParsing finished successfully with no errors!\n
           ");
    }
    else
    {
        printf("\nParsing finished with %d error(s).\n",
           error_count);
    }

    return 0;
}
```

Listing 5: Bison specification with semantic actions

5.5 Makefile

Note: The indentations below must be strict Tabs, not spaces.

```
input = input.txt
output = output.txt

main: lexer.l parser.y
    bison -d parser.y
    flex lexer.l
    gcc parser.tab.c lex.yy.c -o compiler
    ./compiler < $(input) > $(output)
```

Listing 6: Build automation

6 Phase-by-Phase Analysis

6.1 Phase 1: Lexical Analysis

The lexer processes the input character by character and produces the following tokens:

Lexeme	Token Type	Token Code	Attribute
let	Keyword	LET	-
i	Identifier	ID	str_val = ”i”
as	Keyword	AS	-
DOUBLE	Type Keyword	DOUBLE	-
;	Delimiter	SEMI	-
if	Keyword	IF	-
(	Delimiter	LPAREN	-
i	Identifier	ID	str_val = ”i”
==	Operator	EQUOP	-
3	Integer Constant	ICONST	value = 3
)	Delimiter	RPAREN	-
begin	Keyword	BEGINN	-
stop	Keyword	STOP	-
;	Delimiter	SEMI	-
end	Keyword	ENDD	-

Table 1: Token sequence generated by lexical analyzer

Key Observations:

- Total tokens generated: 15
- Keywords identified: let, as, DOUBLE, if, begin, stop, end
- Identifiers: i (appears twice)
- Operators: == (equality comparison)
- Constants: 3 (integer constant)
- Delimiters: ;, (,)
- Whitespace and newlines are consumed but not tokenized

6.2 Phase 2: Syntax Analysis

The parser validates the token sequence against the grammar rules:

Parse Tree Construction:

```
code
 +-- statements
      +-- statements
      |    +-- statement
      |         +-- let_declaration
      |              +-- LET
      |              +-- ID (i)
      |              +-- AS
      |              +-- type
      |              |    +-- DOUBLE
      |              +-- SEMI
      +-- statement
```

```
            +-- if_statement
                 +-- IF
                 +-- LPAREN
                 +-- exp
                 |    +-- exp
                 |    |    +-- ID (i)
                 |    +-- EQUOP
                 |    +-- exp
                 |         +-- constant
                 |              +-- ICONST (3)
                 +-- RPAREN
                 +-- statement
                      +-- begin_block
                           +-- BEGINN
                           +-- statements
                           |    +-- statement
                           |         +-- STOP
                           |         +-- SEMI
                           +-- ENDD
```

Grammar Rules Applied:

1. code → statements
2. statements → statements statement
3. statement → let_declaration
4. let_declaration → LET ID AS type SEMI
5. type → DOUBLE
6. statement → if_statement
7. if_statement → IF LPAREN exp RPAREN statement
8. exp → exp EQUOP exp
9. exp → ID
10. exp → constant
11. constant → ICONST
12. statement → begin_block
13. begin_block → BEGINN statements ENDD

6.3 Phase 3: Semantic Analysis

Semantic analysis performs type checking and symbol table operations:

6.3.1 Symbol Table Operations

Step 1: Variable Declaration

```
Action: Insert variable 'i' with type DOUBLE_TYPE into symbol table
Line: 1
Message: "In line no 1, Inserting i with type DOUBLE_TYPE in symbol table."
```

Symbol Table After Declaration:

Variable Name	Type	Line Declared
i	DOUBLE_TYPE	1

Table 2: Symbol table contents

6.3.2 Type Checking in Expression

Step 2: Analyzing the condition (i == 3)

- **Left operand:** Variable 'i'
  - Check: Is 'i' declared? → YES (found in symbol table)
  - Type: DOUBLE_TYPE
- **Right operand:** Constant '3'
  - Type: INT_TYPE (integer constant)
- **Operator:** == (equality comparison)
- **Type Checking:** typecheck(DOUBLE_TYPE, INT_TYPE)
  - Result: FAIL (type mismatch)
  - Action: Increment error_count
  - Message: "In line no 2, Data type TEXTTTDOUBLE_TYPE is not matched with Data type `INT_TYPE`."

Error Detection:

```
Error Type: TYPE MISMATCH
Location: Line 2
Description: Attempting to compare DOUBLE variable with INT constant
Severity: Semantic Error
```

6.3.3 Semantic Analysis Result

Final Status:

- Total errors: 1
- Error type: Type mismatch in equality comparison
- Variables declared: 1 (variable 'i')
- Variables used: 1 (variable 'i')
- Undeclared variables: 0

7 Execution and Output

7.1 Compilation Process

```
$ make
bison -d parser.y
flex lexer.l
gcc parser.tab.c lex.yy.c -o compiler
./compiler <input.txt> output.txt
```

Listing 7: Build commands

7.2 Generated Files

File	Description
lex.yy.c	Lexical analyzer source (from Flex)
parser.tab.c	Parser source (from Bison)
parser.tab.h	Token definitions and interface
compiler	Final executable program
output.txt	Analysis results

Table 3: Generated files during compilation

7.3 Output File (output.txt)

```
In line no 1, Inserting i with type DOUBLE_TYPE in symbol table.
In line no 2, Data type DOUBLE_TYPE is not matched with Data type
    INT_TYPE.
Parsing finished with 1 error(s).
```

Listing 8: Complete compiler output

8 Detailed Error Analysis

8.1 Error Description

Error Category: Semantic Error (Type Mismatch)

Error Location: Line 2, expression `i == 3`

Problem: The variable `i` is declared as type DOUBLE, but it is being compared with an integer constant `3`. The type checking mechanism detects this incompatibility.

8.2 Why This is an Error

In strongly-typed languages, comparing values of different types without explicit conversion can lead to:

- **Precision loss:** Implicit conversions may lose information

- **Unexpected behavior:** Floating-point comparisons are imprecise
- **Logical errors:** The comparison may not behave as intended

8.3 Possible Solutions

Solution 1: Explicit Type Casting

```
if ((int)i == 3) begin stop;
end
```

Solution 2: Use Floating-Point Constant

```
if (i == 3.0) begin stop;
end
```

Solution 3: Change Variable Type

```
let i as int;
if (i == 3) begin stop;
end
```

9 Symbol Table Implementation

The symbol table (implemented in `symtab.c`) provides essential functions:

9.1 Key Functions

insert(char* id, int type)

- Adds a new identifier to the symbol table
- Records the identifier name and its type
- Prints confirmation message

idcheck(char* id)

- Checks if an identifier has been declared
- Returns 1 if found, 0 otherwise
- Used during semantic analysis to detect undeclared variables

gettype(char* id)

- Retrieves the type of a declared identifier
- Returns type constant (INT_TYPE, DOUBLE_TYPE, etc.)
- Used for type checking in expressions

typecheck(int type1, int type2)

- Compares two types for compatibility
- Returns 1 if types match, 0 otherwise
- Prints error message on type mismatch

10 Compiler Architecture

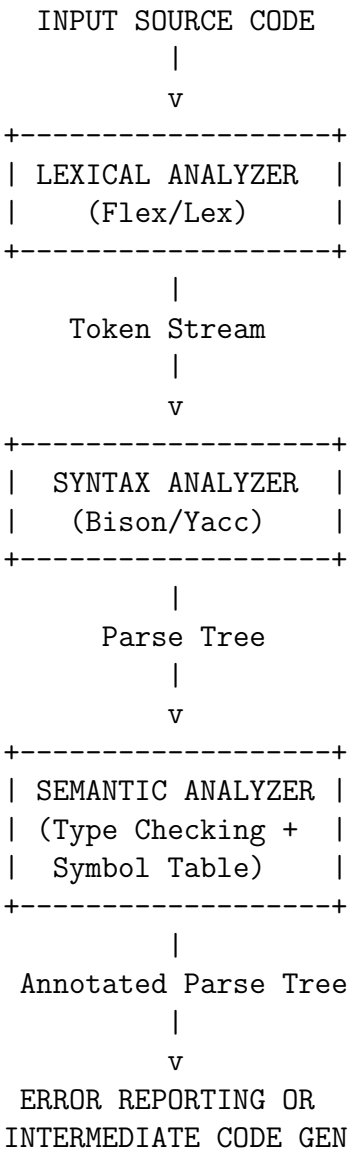

Figure 1: Compiler front-end architecture

11 Key Features Demonstrated

11.1 Lexical Analysis Features

- Pattern matching with regular expressions
- Keyword recognition
- Identifier and constant tokenization
- Comment handling
- Whitespace filtering

- Line number tracking

11.2 Syntax Analysis Features

- Context-free grammar specification
- Recursive statement handling
- Operator precedence and associativity
- Nested block structures
- Grammar rule reduction
- Error recovery mechanisms

11.3 Semantic Analysis Features

- Symbol table management
- Type checking for expressions
- Declaration validation
- Type compatibility verification
- Error counting and reporting
- Semantic action execution

12 Conclusion

This laboratory exercise successfully demonstrated the complete front-end compilation process through three essential phases:

12.1 Achievements

1. **Lexical Analysis:** Successfully tokenized the input code into 15 meaningful tokens including keywords, identifiers, operators, and constants.
2. **Syntax Analysis:** Validated the grammatical structure of the code, constructing a complete parse tree with proper handling of declarations, conditionals, and block structures.
3. **Semantic Analysis:** Implemented type checking and symbol table management, successfully detecting a type mismatch error between DOUBLE and INT types.
4. **Error Detection:** Demonstrated the compiler's ability to identify and report semantic errors with precise line numbers and descriptive messages.
5. **Integration:** Showed how Flex and Bison work together seamlessly to create a functional compiler front-end.